

RAG Chunking Strategy

Evaluation & Benchmark Document

This document is purpose-built to benchmark **Semantic Chunking** versus **Recursive Character Text Splitting** in Retrieval-Augmented Generation (RAG) pipelines. It contains a diverse mix of content types: long-form narrative passages, technical step-by-step instructions, data tables, bullet lists, mid-sentence topic transitions, cross-paragraph entities, dense jargon sections, and intentional ambiguity zones — all structured to expose the trade-offs between the two strategies.

Document Purpose	RAG Chunking Strategy Benchmark
Version	1.0
Content Types	Narrative, Technical, Tabular, Listicle, Mixed
Chunking Strategies Tested	Semantic (embedding-based), Recursive (hierarchical splits)
Recommended Chunk Sizes	256, 512, 1024 tokens
Expected Overlap	0%, 10%, 20%

How to use this document: Run both chunking strategies on this PDF with multiple chunk sizes (256 / 512 / 1024 tokens) and overlap settings (0% / 10% / 20%). For each retrieved chunk, evaluate: (1) context completeness, (2) topic coherence, (3) boundary quality, and (4) entity preservation. Sections are labeled by the challenge they introduce.

Section 1: Extended Narrative Passage

Challenge: Semantic topic drift within a single paragraph block

This section is designed to test how well each chunking strategy detects natural semantic boundaries in flowing prose where topic shifts occur mid-paragraph or mid-sentence.

The history of information retrieval stretches back to the early days of library science, when Melvil Dewey introduced his decimal classification system in 1876. Librarians spent decades developing controlled vocabularies and index cards to help patrons locate relevant materials. The fundamental challenge was always the same: how do you match a user's information need to the right document when neither is expressed in identical language? Boolean keyword search, introduced in digital systems during the 1970s, improved speed dramatically but remained brittle — a query for 'heart attack' would miss documents using the term 'myocardial infarction.'

The vector space model, proposed by Salton, Wong, and Yang in 1975, changed everything. By representing documents and queries as vectors in high-dimensional space, retrieval systems could now compute similarity scores rather than relying on exact term matches. TF-IDF weighting gave more importance to rare, distinctive terms and less to common stopwords. These foundations proved robust enough to power web search engines through the late 1990s. Meanwhile, in a completely different domain, molecular biologists were wrestling with protein sequence databases. The BLAST algorithm, developed in 1990 at the NCBI, faced strikingly similar challenges: how do you efficiently find a short query sequence within a corpus of billions of base pairs? The heuristic approach BLAST used — seeding with short exact matches, then extending — would later inspire ideas in approximate nearest neighbor search used in modern vector databases.

Dense retrieval fundamentally altered the landscape again in 2020. The DPR (Dense Passage Retrieval) paper from Facebook AI demonstrated that bi-encoder models fine-tuned on question-answer pairs could dramatically outperform BM25 on open-domain question answering benchmarks. The key insight was deceptively simple: encode the question and the passage independently with BERT-style transformers, then retrieve by maximum inner product search. This decoupling allowed offline indexing of the entire corpus — a crucial engineering advantage. What is less often discussed is that DPR's success was partly an artifact of the Natural Questions dataset on which it was evaluated. Natural Questions queries tend to be factoid-style, seeking named entities or short factual answers. On tasks requiring multi-hop reasoning or nuanced synthesis, sparse retrieval methods like BM25 often remain competitive or superior. The lesson: no single retrieval paradigm dominates across all query distributions.

Hybrid retrieval systems — combining dense and sparse signals — have emerged as the practical consensus for production RAG systems as of 2024. Reciprocal Rank Fusion (RRF) and learned sparse retrieval methods like SPLADE offer different paths to integration. The chunking strategy upstream of retrieval, however, remains an underexplored variable. A perfectly tuned hybrid retriever can still fail if the chunks it indexes split a critical explanation across two boundaries, losing the connective logic that makes the answer coherent. This is exactly the failure mode this document is designed to surface.

Section 2: Technical Step-by-Step Instructions

Challenge: Numbered steps that span page/chunk boundaries

Recursive splitters can easily fragment numbered instructions because they split on newlines without considering procedural continuity. Semantic splitters should keep causally linked steps together.

2.1 Setting Up a ChromaDB Vector Store for RAG

Step 1 — Install dependencies. Begin by creating a fresh virtual environment: `python -m venv .venv && source .venv/bin/activate`. Install the required packages: `pip install chromadb sentence-transformers langchain`. Verify the installation by running `python -c "import chromadb; print(chromadb.__version__)"`. You should see version 0.4.x or higher.

Step 2 — Initialize the client and collection. For persistent storage, use `chromadb.PersistentClient(path='./chroma_db')`. For in-memory testing, use `chromadb.Client()`. Create a collection with a distance metric appropriate for your embedding model: cosine similarity works well for most sentence-transformer models, while dot product is preferred for normalized OpenAI ada-002 embeddings.

Step 3 — Choose and load your embedding model. The choice of embedding model has a larger impact on retrieval quality than the choice of chunking strategy. For English-only use cases, BAAI/bge-base-en-v1.5 offers a strong quality-speed trade-off. For multilingual applications, `intfloat/multilingual-e5-base` is recommended. Load via `SentenceTransformer('BAAI/bge-base-en-v1.5')`.

Step 4 — Chunk your documents. This is the step this document is designed to evaluate. For recursive chunking, use LangChain's `RecursiveCharacterTextSplitter` with `chunk_size=512`, `chunk_overlap=50`. For semantic chunking, use `SemanticChunker` with a percentile breakpoint threshold of 0.75-0.85. Always inspect 10-20 chunks manually before indexing at scale — bad splits are invisible until retrieval fails.

Step 5 — Generate embeddings and upsert. Batch your chunks into groups of 32-64 before embedding to avoid memory issues with large documents. Pass each batch through your embedding model, then call `collection.add(embeddings=..., documents=..., ids=...)`. Always include source metadata — file name, page number, chunk index — in the `metadatas` parameter. This is critical for citation and debugging.

Step 6 — Query and evaluate. Test retrieval with diverse query types: factoid ('What year was X introduced?'), conceptual ('How does Y work?'), comparative ('What is the difference between A and B?'), and multi-hop ('Given that X causes Y, what happens when Z occurs?'). Measure `MRR@10`, `Recall@5`, and `NDCG@10`. Pay special attention to multi-hop queries — these expose chunking failures most dramatically because the answer requires connecting information that a naive splitter may have placed in non-adjacent chunks.

NOTE: Steps 4 and 5 are intentionally verbose because poor chunkers frequently split between the 'chunk your documents' instruction and the parameters that follow it, creating an incomplete chunk that answers 'what tool?' but not 'with what settings?'

Section 3: Comparative Data Tables

Challenge: Tabular context loss when table is split across chunks

Tables are the nemesis of recursive splitters. A table header split from its rows creates chunks that are individually meaningless. Semantic chunkers should treat tables as atomic units.

Table 3.1 — Chunking Strategy Comparison Matrix

Property	Recursive Splitting	Semantic Chunking	Winner
Speed (indexing)	Very Fast	Slow (needs embeddings)	Recursive
Boundary Quality	Poor (char/newline-based)	Good (meaning-based)	Semantic
Chunk Size Consistency	High (deterministic)	Low (variable)	Recursive
Entity Preservation	Poor	Good	Semantic
Table Handling	Breaks tables	Keeps tables intact*	Semantic
Code Block Handling	Splits at indentation	Usually intact	Semantic
Multi-language Support	Native	Depends on model	Recursive
Cost	Zero	Embedding API cost	Recursive
Interpretability	High	Low (black box)	Recursive
Best For	Homogeneous prose docs	Mixed-content, QA tasks	—

*Semantic chunking preserves tables only when using PDF parsers that maintain table structure.

Table 3.2 — Benchmark Results: Natural Questions Dataset (Recall@5)

Method	Chunk: 256	Chunk: 512	Chunk: 1024	Notes
BM25 + Recursive (0% overlap)	0.61	0.67	0.70	Baseline
BM25 + Recursive (10% overlap)	0.63	0.69	0.71	+overlap helps
Dense + Recursive (0% overlap)	0.71	0.76	0.74	Peak at 512
Dense + Recursive (20% overlap)	0.73	0.78	0.75	Overlap +2pp
Dense + Semantic (percentile 0.80)	0.74	0.81	0.81	Best at 512+
Dense + Semantic (percentile 0.90)	0.69	0.79	0.82	High threshold
Hybrid + Recursive (RRF)	0.75	0.80	0.78	
Hybrid + Semantic (RRF)	0.76	0.83	0.83	Best overall

Benchmark figures are illustrative approximations based on published literature. Bold row = best overall performance.

Section 4: Mixed-Topic Passage (Intentional Ambiguity)

Challenge: Three topics interleaved — semantic chunker must decide boundaries

The following passage intentionally blends three separate topics without clear headers or paragraph breaks between them. This is common in real-world documents like meeting notes, research summaries, and scraped web content. This passage will challenge **both** strategies: recursive splitters will cut mid-argument; semantic splitters may fail to resolve the ambiguous transitions.

Transformer attention mechanisms compute scaled dot-product attention over query, key, and value matrices derived from the input token embeddings. The attention score matrix has dimensionality $O(n^2)$ with respect to sequence length, which creates a quadratic compute bottleneck for long documents — a 10,000-token document requires 100 million attention computations per layer. This is why naive RAG pipelines chunk documents before embedding rather than feeding full documents to a cross-encoder. Separately, the legal implications of training large language models on copyrighted internet text remain unresolved as of 2025. The Authors Guild, the New York Times, and various individual authors have filed suits against major AI labs arguing that ingestion without license constitutes infringement. The fair use defense most labs rely on — transformative use for the purpose of building new technology — has not been fully adjudicated in the context of foundation models. These suits are proceeding in parallel in multiple jurisdictions, creating regulatory uncertainty that affects enterprise adoption of RAG systems that index proprietary content. Meanwhile, microservices architectures have fundamentally changed how RAG systems are deployed. Rather than a monolithic pipeline where ingestion, embedding, indexing, retrieval, and generation all run in the same process, production systems split these into independently scalable services. The ingestion service handles document parsing and chunking; the embedding service wraps the encoder model behind an HTTP API; the retrieval service maintains the vector index and serves nearest-neighbor queries; the generation service orchestrates prompts and calls the LLM. This separation enables independent scaling — during batch indexing, you scale the embedding service horizontally; during peak query load, you scale the retrieval and generation services instead.

Section 5: Dense List Content

Challenge: Recursive splitters often treat each bullet as an independent chunk, losing the unifying context (the heading/topic) that makes the list meaningful.

5.1 Key Parameters to Tune in Semantic Chunking

- **Embedding model choice** — The semantic similarity signal is only as good as the embeddings. A domain-specific model (e.g., `medbert` for clinical text) will produce better semantic boundaries than a general-purpose model.
- **Breakpoint percentile / threshold** — The cosine similarity drop threshold determines how sensitive the chunker is to topic shifts. Lower values (0.70) create many small chunks; higher values (0.95) create fewer, larger chunks. Tune on a held-out validation set, not on the test set.

- **Minimum and maximum chunk size guardrails** — Without guardrails, semantic chunking can produce single-sentence chunks (too granular) or 3,000-token chunks (too broad). Set hard limits: min 100 tokens, max 800 tokens as a starting point.
- **Sliding window buffer size** — Some implementations compute similarity over a window of consecutive sentences rather than adjacent pairs. A buffer size of 2-3 sentences reduces noise from stylistic variations within a single topic.
- **Pre-processing quality** — Semantic chunking is more sensitive to noisy input than recursive splitting. Remove headers, footers, page numbers, and watermark text before chunking; these inject spurious semantic signals that confuse the boundary detector.
- **Overlap strategy** — Semantic chunks rarely need explicit overlap because boundaries are chosen to be informationally complete. However, adding a small overlap (1-2 sentences) at the end of each chunk can help with queries that reference information spanning adjacent semantic units.

5.2 Key Parameters to Tune in Recursive Character Text Splitting

- **Separator hierarchy** — The default LangChain hierarchy is `["\n\n", "\n", " ", ""]`. For Markdown documents, prepend `["## ", "### ", "\n\n"]`. For Python code, use `["\nclass ", "\ndef ", "\n\n"]`.
- **chunk_size** — Measured in characters by default; set in tokens using a tokenizer-aware length function. Always use token counts when working with an LLM that has a token budget for context.
- **chunk_overlap** — Start with 10-15% of `chunk_size`. Higher overlap increases storage cost and index size but improves recall on boundary-spanning queries. For chunked at 512 tokens, 50-75 tokens overlap is a common production setting.
- **length_function** — Use `tiktoken` or the target LLM's tokenizer rather than Python's `len()`. A chunk of 500 ASCII characters may be 120 tokens or 350 tokens depending on content.
- **is_separator_regex** — Setting this to `True` allows complex split patterns. Useful for splitting on sentence boundaries using `r'(?<=[.!?])\s'` as the separator.

Section 6: Entity and Cross-Reference Stress Test

Challenge: Entities introduced early in the document are referenced without re-introduction later. Chunks that don't contain the introduction are semantically incomplete.

Project Helios is Acme Corp's internal RAG system, first deployed in Q1 2023 to serve the legal and compliance teams. The system ingests 40,000 documents per week from 17 internal repositories. The primary embedding model is text-embedding-3-large from OpenAI, with a secondary bge-m3 model handling non-English documents (primarily German and Japanese contracts). The vector database is a managed Qdrant cluster running on AWS us-east-1.

In Q3 2023, the Helios team observed a 23% drop in user-rated answer quality after migrating the document ingestion pipeline from a custom regex-based splitter to LangChain's RecursiveCharacterTextSplitter. Investigation revealed that the new splitter was fragmenting multi-page contract clauses that the old splitter had preserved as atomic units by splitting only on page breaks. The retrieval system was returning clause fragments without the preamble that defined the governing law or the parties — rendering the retrieved text legally ambiguous.

The Helios team tested three remediation strategies. First, they increased chunk_overlap from 0 to 200 tokens. This improved the boundary-spanning query scores by 11% but increased index size by 28% and caused noisy duplicate retrievals. Second, they implemented a hybrid splitter: recursive splitting within sections, but forced breaks only at defined structural markers (clause headers in the contract template). This improved scores by 19% but required a custom parser for each document template. Third, they migrated to semantic chunking using an embedding model fine-tuned on legal text (legal-bert-base-uncased). This approach achieved a 31% improvement in answer quality scores with no increase in index size, but tripled ingestion latency. Helios ultimately adopted strategy three for new document types and maintained the legacy regex splitter for document types already well-served by the old pipeline.

The following passage references Helios again without re-introduction — a deliberate test of whether a retrieved chunk remains coherent without the preceding context:

As of Q2 2025, Helios processes over 2 million queries per month. The p50 latency is 340ms end-to-end; p99 latency is 1.8 seconds, driven primarily by the LLM generation step rather than retrieval. The engineering team has begun evaluating late interaction models (ColBERT-v2) as a replacement for the bi-encoder retrieval stage, with early results showing 8% MRR improvement on the internal evaluation set at 4x the storage cost.

Section 7: Dense Technical Specification

Challenge: High information density — recursive splitters create chunks that are each syntactically complete but semantically incomplete without adjacent chunks.

7.1 HNSW Index Parameters for Production RAG

Hierarchical Navigable Small World (HNSW) graphs are the dominant approximate nearest neighbor index structure in production vector databases. The following parameters govern the quality-speed trade-off:

Parameter	Type	Default	Production Rec.	Effect
M	int	16	32–64	Graph connectivity. Higher = better recall, more RAM.
ef_construction	int	200	400–800	Build-time beam search width. Higher = slower build, better index.
ef_search	int	100	100–400	Query-time beam width. Tune per latency SLA.
max_elements	int	auto	Set 2x expected	Pre-allocates memory. Resize is expensive.
ml	float	$1/\ln(M)$	default	Level generation factor. Rarely tuned.

7.2 Embedding Dimensionality and Quantization

Full-precision (float32) embeddings for text-embedding-3-large at 3072 dimensions require 12KB per vector. At 10 million chunks, this is 120GB of raw embedding storage before index overhead. Three quantization strategies trade accuracy for storage savings:

- **Scalar quantization (int8):** Reduces storage 4x (3GB per 10M vectors). Recall degradation typically <1% on general QA benchmarks. Supported natively in Qdrant, Weaviate, and Pinecone.
- **Binary quantization:** Reduces storage 32x (375MB per 10M vectors). Requires oversampling and re-ranking with full-precision vectors — retrieve 10x candidates with binary index, re-rank with float32. Net recall close to full precision at 5–10x the query throughput.
- **Product quantization (PQ):** Flexible compression ratio (8x–64x). Encodes each vector as a sequence of codebook indices. Training the codebook requires a representative sample of your corpus. Best for very large (100M+) corpora where binary quantization recall is insufficient.
- **Matryoshka Representation Learning (MRL):** A training technique rather than a post-hoc quantization method. MRL models (e.g., OpenAI's text-embedding-3 family) can be truncated to lower dimensions at query time with minimal accuracy loss. Truncating to 256 dimensions from 3072 achieves ~93% of full-dimension recall at 12x the storage savings and 12x the query speed.

Section 8: Abrupt Mid-Sentence Topic Transitions

Challenge: Sentences that contain two distinct concepts — semantic chunkers must decide whether to split or keep them together.

The following passage contains sentences that pivot topics mid-clause. This pattern appears frequently in executive summaries, abstracts, and scraped blog content. Neither chunking strategy handles it cleanly — but they fail in different, diagnostically distinct ways.

Although the transformer architecture was originally designed for machine translation, the coffee market in Ethiopia saw a 14% export increase in 2023 driven by increased demand from South Korean specialty roasters who, coincidentally, also fund a significant portion of AI research through the Samsung Global Research network. Vector databases, which store embeddings in a format optimized for cosine similarity search, are now used in applications ranging from drug discovery — where molecular fingerprints are encoded as vectors — to fraud detection systems at major European banks, which operate under GDPR constraints that complicate the storage of raw transaction embeddings and have therefore pushed several institutions toward on-premise deployment of open-source vector databases like Qdrant and Milvus. The EU AI Act, scheduled to come into full effect in 2026, introduces additional compliance layers for 'high-risk' AI systems, a categorization that likely includes RAG systems used in medical diagnosis, legal advice, or credit decisions, though the exact regulatory perimeter is still being defined by national competent authorities in Germany, France, and Sweden, each of which has published preliminary guidance that diverges on several key interpretive questions.

The above paragraph is deliberately adversarial. A good semantic chunker should either (a) refuse to split it because no clean semantic boundary exists, or (b) split after 'South Korean specialty roasters' and again after 'open-source vector databases like Qdrant and Milvus.' A recursive splitter will likely split at a mid-clause character boundary, producing fragments that are syntactically broken.

Section 9: FAQ Format Content

Challenge: Q&A; pairs should ideally be chunked together. Recursive splitters frequently split the answer from its question.

Q: What is the difference between chunking and splitting in the context of RAG?

A: In practice, the terms are used interchangeably, but 'splitting' typically refers to the mechanical act of dividing text by a rule (character count, separator match), while 'chunking' implies a more deliberate strategy that considers semantic or structural units. All splitting strategies produce chunks, but not all chunking strategies rely on simple splits.

Q: Does chunk size affect the quality of generated answers, or only retrieval recall?

A: Both. Smaller chunks improve retrieval precision — the retrieved text is more tightly focused on the query topic — but reduce answer completeness because the chunk may not contain the full explanation. Larger chunks improve completeness but dilute relevance and may exceed the LLM's effective context window. The optimal chunk size is task-dependent and must be tuned empirically.

Q: Is semantic chunking always better than recursive chunking?

A: No. Semantic chunking is superior for mixed-content documents, long-form prose, and QA tasks where query-answer pairs span complex semantic units. Recursive chunking remains competitive on homogeneous, well-structured documents (e.g., API documentation with consistent heading hierarchies) and is strongly preferred when ingestion latency and cost are binding constraints. The correct answer is almost always: benchmark both on your specific corpus and query distribution.

Q: Can I combine semantic and recursive chunking?

A: Yes, and this is increasingly the recommended approach. Use structural markers (headers, sections, page breaks) for a coarse first-pass split, then apply semantic chunking within each coarse unit. This constrains the semantic chunker to operate over manageable spans and prevents it from accidentally merging content from unrelated sections that happen to have similar surface-level embeddings.

Q: How do I evaluate chunking quality without running a full RAG pipeline?

A: Use the chunk coverage metric: given a set of known (question, answer-span) pairs, measure what fraction of answer spans are fully contained within a single chunk. A span that is split across two chunks will never be retrieved in full by a single-chunk retrieval step. This metric is fast to compute and correlates well with end-to-end RAG performance on factoid QA tasks.

Section 10: Code Blocks and Mixed Code/Prose

Challenge: Code blocks split mid-function are syntactically and semantically broken. Both strategies should treat functions as atomic units.

The following code implements a simple semantic chunker from scratch using sentence-transformers and cosine similarity. It is provided both as a functional reference and as a chunking test case — the function body should never be split from its docstring or its return statement.

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def semantic_chunk(text: str, model_name: str = 'BAAI/bge-base-en-v1.5',
                  percentile_threshold: float = 0.80,
                  min_tokens: int = 100, max_tokens: int = 800) -> list[str]:
    """
    Split text into semantically coherent chunks using cosine similarity
    between consecutive sentence embeddings.

    Args:
        text: Input document text.
        model_name: SentenceTransformer model identifier.
        percentile_threshold: Similarity drop percentile that triggers a split.
        min_tokens: Minimum chunk size in whitespace-tokenized words.
        max_tokens: Maximum chunk size in whitespace-tokenized words.

    Returns:
        List of chunk strings.
    """
    model = SentenceTransformer(model_name)
    sentences = [s.strip() for s in text.split('.') if s.strip()]
    embeddings = model.encode(sentences, batch_size=32, show_progress_bar=False)

    similarities = []
    for i in range(len(embeddings) - 1):
        sim = cosine_similarity([embeddings[i]], [embeddings[i + 1]])[0][0]
        similarities.append(sim)

    threshold = np.percentile(similarities, (1 - percentile_threshold) * 100)
    split_indices = [i for i, s in enumerate(similarities) if s < threshold]

    chunks, current = [], []
    for i, sentence in enumerate(sentences):
        current.append(sentence)
        if i in split_indices:
            chunk_text = '. '.join(current) + '.'
            word_count = len(chunk_text.split())
            if word_count >= min_tokens:
                chunks.append(chunk_text)
                current = []
    if current:
        chunk_text = '. '.join(current) + '.'
        word_count = len(chunk_text.split())
        if word_count >= min_tokens:
            chunks.append(chunk_text)
```

```

        chunks.append(chunk_text)
        current = []
    if current:
        chunks.append(' '.join(current) + '.')
    return chunks

```

The function above is 38 lines long. At 256-token chunk size with no overlap, a recursive splitter will almost certainly split it between the docstring and the function body, and again before the return statement. At 512 tokens, it may survive intact. At 1024 tokens, it will always survive. This predictable behavior is why 512 tokens is the most common default in production systems — it is a rough empirical optimum for code-containing documents.

Section 11: Conclusion and Evaluation Checklist

This document has presented eleven distinct content patterns, each designed to stress-test a different aspect of chunking strategy behavior. The table below summarizes the expected failure modes for each section under each strategy.

Section	Content Type	Recursive Failure Mode	Semantic Failure Mode
\$1	Long narrative	Mid-paragraph splits	May merge sub-topics
\$2	Step-by-step instructions	Steps split from params	Usually intact
\$3	Data tables	Table header/row split	Intact if parser used
\$4	Mixed-topic prose	Mid-argument splits	Confused boundaries
\$5	Bullet lists	Each bullet as chunk	Missing heading context
\$6	Entity cross-references	Dereference failure	Dereference failure
\$7	Dense technical spec	Spec fragments	Usually intact
\$8	Abrupt topic pivots	Mid-clause breaks	Over-merges topics
\$9	Q&A pairs	Q/A separation	Usually keeps Q+A
\$10	Code blocks	Mid-function splits	Usually intact

Evaluation Checklist

For each chunking strategy and each chunk size (256 / 512 / 1024 tokens):

- [] Count chunks that split numbered steps from their parameters (\$2 failure)
- [] Count chunks with orphaned table rows or missing headers (\$3 failure)
- [] Count Q&A; pairs where question and answer are in separate chunks (\$9 failure)
- [] Count code function definitions that are split mid-body (\$10 failure)
- [] Query: 'What is Project Helios?' – check if chunk includes Q3 2023 context (\$6)

- [] Query: 'What is the ef_construction parameter?' – check if table row is complete (§7)
- [] Query: 'How do I set up a ChromaDB vector store?' – check if all 6 steps retrieved
- [] Compute chunk coverage score on all (Q, answer-span) pairs in this document